Interview Prep — dbt Semantic Layer Governance

> *Study material for the pms-semantic-layer project. This is the study/interview-prep track; the source project holds the actual code, dbt models, and blog posts this document studies. Continues in 02-agent-routing-and-scope.md.*

Same format as the interview-prep reference document the project follows: a question, the weak/tempting answer, the gap in it, and the correct answer -- plus what to study if this topic comes up cold in a real interview. Covers chapter 10 (docs, schemas, exposures) and chapter 11 (contracts, freshness) as they were actually built, not in the abstract.

---

1. `{% docs %}` blocks vs inline `description:`

Q: When would you use a `{% docs %}` block instead of just writing the description inline in the YAML?

- **Weak answer**: "For long text." (True, but incomplete -- doesn't explain the actual engineering reason.)
- **The gap**: length isn't the real driver. The real reason is *reuse and single-sourcing*. If the same explanation applies to more than one column, or you want other docs to reference it (`{{ doc('adr') }}` pointing at the same underlying revenue-proration story as `revenue`'s own doc), inline strings force you to duplicate text that will drift out of sync the first time someone updates one copy and not the other.
- **Correct answer**: use `{% docs %}` for anything reusable or long enough that you want a single source of truth; use inline `description:` for short, one-off, non-reused facts (identifiers, simple formulas).
- **Study**: dbt docs on doc blocks, how `dbt docs generate` resolves `{{ doc(...) }}` at compile time (it's just Jinja -- no magic).

---

2. Why documentation lives with a data model at all

Q: Isn't documentation just for humans? Why does it matter for an AI agent querying the warehouse?

- **Weak answer**: "So the agent has more context in its prompt."
- **The gap**: this treats docs as decoration. In this project, every doc block we wrote exists because a wrong number already happened once (revenue not prorated, `booking_window` computed from `created_at`). Documentation here is a **record of a previously wrong assumption**, written down specifically so it can't be silently repeated -- by a human OR a model doing text-to-SQL.
- **Correct answer**: documentation is a defense against the exact failure mode from chapter 07 (an LLM confidently producing a plausible-but-wrong number). A semantic layer's docs are most valuable exactly where the "obvious" interpretation of a column name is wrong.
- **Study**: re-read chapter 07's silent-hallucination case ("occupancy rate: 70%", no query run) -- that's the failure this documentation work is a defense against.

---

3. Schema separation (`generate_schema_name` override)

Q: dbt already lets you set `+schema:` per folder. Why did you also need to touch a macro?

- **Weak answer**: "To set the schema name." (Doesn't explain the default behavior being overridden.)

- **The gap**: dbt's default `generate_schema_name` macro **concatenates** the target's schema with your custom one (`dev_staging` , not `staging`) -- a deliberate default so multiple developers sharing one Postgres instance don't collide on the same schema name. If you don't know this default exists, you'll be confused the first time your schema doesn't look like what you typed.
- **Correct answer**: override `generate_schema_name` (a macro you can redefine because it's just a Jinja macro dbt calls by convention) to special-case a `prod` target to use the exact name, while every other target keeps the safety prefix. This is the dbt Labs-documented pattern, not a project-specific hack.
- **Study**: dbt docs: custom schema macro, and the concept of macro override-by-name (no explicit "override" keyword -- last definition wins, project macros beat package macros).

Q: Why does schema separation matter for security, not just tidiness?

- **Weak answer**: "It's cleaner."
- **The gap**: staging is an uncurated, 1:1 mirror of raw sources -- no business rules, no proration, no NULL handling guarantees. If a BI tool or an agent can query staging directly, every fix we've made in the mart layer (proration, `NULLIF` , cancellation filtering) can be silently bypassed.
- **Correct answer**: separate schemas exist so `GRANT SELECT` can target `marts` alone -- staging is implementation detail, not something any external consumer should have a connection string for.

---

4. Exposures

Q: What's the difference between an exposure and a regular dbt model dependency?

- **Weak answer**: "It shows what depends on a model." (Vague -- doesn't distinguish it from `ref()` .)
- **The gap**: `ref()` tracks dependencies *within* the dbt DAG (model → model). An exposure tracks a dependency **dbt can't see any other way** -- something outside the DAG entirely (a dashboard, a notebook, an application) that reads the built tables directly, via SQL dbt never compiled.
- **Correct answer**: exposures exist so `dbt run` / `dbt build` with `--select state:modified+` (or similar) can flag "this change affects `text_to_sql_agent` " even though the agent's code lives entirely outside the dbt project and dbt has never executed a line of it.
- **Study**: dbt docs: exposures, and the `maturity` field's actual purpose (signals confidence/stability of the consumer, not of the data).

Q: We wrote an exposure for an agent that doesn't exist yet. Is that a legitimate use, or documentation theater?

- **Weak answer**: "It's fine, more docs are always good."
- **The gap**: an exposure for nothing real just adds noise. The legitimate reason here is specific: chapters 12-13 are *already scoped and committed to* in this project's plan, and the exposure's job is to make a future schema change to `mart_kpi` / `mart_aggregates` show up as a breaking-change warning against a named consumer *before* that consumer's code is written -- which is exactly when a breaking change is cheapest to catch.
- **Correct answer**: exposures for not-yet-built consumers are legitimate when the consumer is a committed, scoped piece of near-future work -- not for hypothetical "maybe someday" integrations.

---

5. Data contracts (`contract: {enforced: true}`)

Q: We already have 53 passing dbt tests. Why isn't that enough to guarantee a mart's shape?

- **Weak answer**: "Tests check more things, like value ranges." (True in general, but misses the actual mechanism gap.)

- **The gap**: tests run **after** a model is built, against the resulting data -- they validate content, not structure. A `not_null` test on a column that silently became `TEXT` instead of `NUMERIC` will still pass (a non-null string is still non-null). Nothing about a normal test suite catches a type change unless you specifically test for the type itself.
- **Correct answer**: `contract: {enforced: true}` moves the check to **before** the table is built. dbt compares the declared `data_type` for every column against what the compiled query actually produces, and fails the run outright on any mismatch -- verified live in this chapter by deliberately casting `occupied_nights` to `TEXT` and watching `dbt run` refuse to create the table, citing the exact column and type mismatch.
- **Study**: dbt docs: model contracts. Know that contracts require **every** column to have a declared `data_type` (not just the ones you care about) and that column order/names become part of the enforced shape too.

Q: dbt warned about "unspecified precision/scale" on a `numeric` column even though the contract passed. What does that mean, and should you fix it?

- **Weak answer**: "It's just a warning, ignore it."
- **The gap**: Postgres' bare `numeric` type has no fixed precision/scale -- it's a valid type, but two different queries could both produce "numeric" values with wildly different rounding behavior, and the contract's type-equality check wouldn't catch a change there (still `numeric` either way). The warning exists because "the type check passed" and "the actual precision is what you expect" are two different guarantees.
- **Correct answer**: for a column that flows into a mart meant to be trusted (revenue, in this project), declare the exact precision/scale (`numeric(10,2)`) so both the contract *and* the underlying Postgres column definition are locked down -- not just the type family.

Q: Why apply the contract to `mart_aggregates` and not `mart_kpi` in this project?

- **Weak answer**: "Because we ran out of time." (A cop-out, even if partially true.)
- **The gap**: the actual reasoning that was applied: `mart_kpi` is built *on top of* `mart_aggregates` , so a contract on the upstream model has more leverage -- it protects every downstream consumer transitively. `mart_kpi` 's columns are also all derived (`round()` , division), which raises legitimate open questions about precision choices per KPI that weren't resolved yet (e.g. `avg_booking_window_days` rounds to 1 decimal, others to 2) -- a real, stated reason to defer, not an oversight.
- **Correct answer**: contracts don't have to be applied everywhere at once. Start at the model with the most downstream leverage, and only add a contract elsewhere once the precision/type decisions for *that specific model* have actually been made deliberately.

---

6. Source freshness

Q: What does `dbt source freshness` actually check?

- **Weak answer**: "Whether the data is up to date."
- **The gap**: too vague to be useful in an interview -- it doesn't check *content* at all. It's a single comparison: `now() - loaded_at_field` on the most recent row, against `warn_after` / `error_after` thresholds. It says nothing about whether the data is correct, only about how long ago the source last received a write.
- **Correct answer**: it's a staleness check, not a correctness check -- a source can pass freshness and still be full of garbage, or fail freshness while every row in it is perfectly valid (just old).
- **Study**: dbt docs: source freshness.

Q: For the `reservation` source, why `updated_at` and not `created_at` -- doesn't this project usually warn against trusting technical timestamps?

- **Weak answer**: "updated_at is more recent." (True but doesn't explain the actual selection criterion.)
- **The gap**: this is the one place in the project where the instinct from chapters 09/10 (avoid technical timestamps, prefer real business dates like `booked_at` / `listed_at`) would lead you wrong if applied blindly. Freshness isn't measuring a *business* fact (when did the guest book) -- it's measuring an *operational* fact (when did this table last get a write), which is exactly what a technical timestamp is for.
- **Correct answer**: `updated_at`, because it's the one column proven (not assumed) to move on every write, including cancellations -- verified by grepping the application code for where it's set (`reservation_engine.py`, `UPDATE ... SET updated_at = now()` on cancellation) rather than trusting the column name and a `DEFAULT now()` alone, since a `DEFAULT` only fires on `INSERT` and would silently freeze on `UPDATE` without an explicit assignment or a trigger.
- **Correct general rule**: business-date columns and technical-timestamp columns aren't "good" or "bad" in isolation -- they're the right or wrong tool depending on whether the question is "what happened, from the business's perspective" (use `booked_at` / `listed_at`) or "when did this row last change" (use `created_at` / `updated_at`).

Q: How do you pick `warn_after` / `error_after` thresholds instead of guessing?

- **Weak answer**: pick round numbers that feel safe (e.g. "1 hour", "1 day").
- **The gap**: an arbitrary threshold either fires constantly on normal variance (too tight) or takes hours to notice a real outage (too loose). The actual write cadence of the system producing the data is the only real anchor.
- **Correct answer**: go find the real write interval before choosing anything -- in this project, `app_development/scripts/simulate.py`'s `INTERVAL_SECONDS` (default 5s) is the producer's actual cadence. Thresholds should be a multiple of that cadence large enough to absorb normal jitter (not every 5s tick produces a write -- some attempts fail business rules) but small enough to catch a real outage quickly.

Q: What's the operational difference between a source hitting `warn_after` vs `error_after`?

- **Weak answer**: "One shows a warning, one shows an error."
- **The gap**: doesn't answer what actually happens to the *process*. `dbt source freshness` exits with status code 0 on a WARN, and a non-zero exit code on an ERROR STALE result -- which is the actual mechanism that matters in a CI/orchestration context (a `1` exit code fails a pipeline step and can block anything scheduled after it; a `0` exit code lets the pipeline continue with just a logged warning).
- **Correct answer**: verified live in this project -- the `simulator` container had actually been stopped for 7 days (a real, pre-existing staleness, not staged), so `dbt source freshness --select source:pms.reservation` returned `ERROR STALE` with exit code `1`. After restarting the simulator, the same command returned `PASS` with exit code `0`. `error_after` is a hard gate; `warn_after` is a signal that doesn't stop anything downstream by itself.
- **Study**: check exit codes with `echo $?` immediately after the command -- piping through another command (`| tail`) captures *that* command's exit code, not dbt's, which is an easy mistake to make when demoing this.

---

7. A data test failure found by accident, not staged

Q: How do you tell the difference between an invariant test that's too strict, and one that just caught a real bug?

- **Weak answer**: "If the test fails, there's a bug."
- **The gap**: the actual investigative step matters more than the conclusion. Restarting the simulator (to demonstrate source freshness recovering from `ERROR STALE` to `PASS`) caused `dbt test` to fail a pre-existing guard (`available_nights >= occupied_nights`) that had been green for the entire project up to this point -- a real regression appearing live, not staged for the chapter.
- **Root cause, found by tracing data, not guessing**: `accommodation.listed_at` was backfilled once (chapter 09, migration 011) from the reservations that existed *at that time*. `app_development/scripts/reservation_attempts.py` 's `random_date_range()` generates new reservations within a fixed rolling window (`today − 365d` to `today + 90d`) with **no awareness of `listed_at` at all** -- so as the simulator keeps running for days after the backfill, it can (and did) generate a reservation whose `start_date` falls before its own accommodation's `listed_at`, silently violating an invariant that had only ever held by historical coincidence, not by any enforced rule.
- **The fix**: pass each accommodation's `listed_at` into `random_date_range()` and clamp the window's lower bound to `max(today − 365d, listed_at)` -- fixing the generator at its root, not patching the symptom in the mart. Verified the fix by checking `docker inspect ... StartedAt` for the exact moment the corrected container came online, then confirming zero violations among reservations created after that timestamp -- not just "the test is green now," which wouldn't have distinguished a real fix from lucky timing.
- **The rule**: a one-time backfill that establishes an invariant (`listed_at <= every future reservation.start_date for this accommodation`) is not the same as an invariant that's *enforced* going forward. If the thing that violates it is a system you don't control at the schema level (a data generator, an external producer), the fix belongs in that producer, not in a downstream mart or a wider test tolerance.
- **Study**: this is a live example of the "backfill vs. ongoing invariant" gap -- the same category of bug as `booked_at` / `listed_at` in chapter 09, but this time surviving *past* the fix that was supposed to close it, because the fix addressed historical data without constraining future data generation.